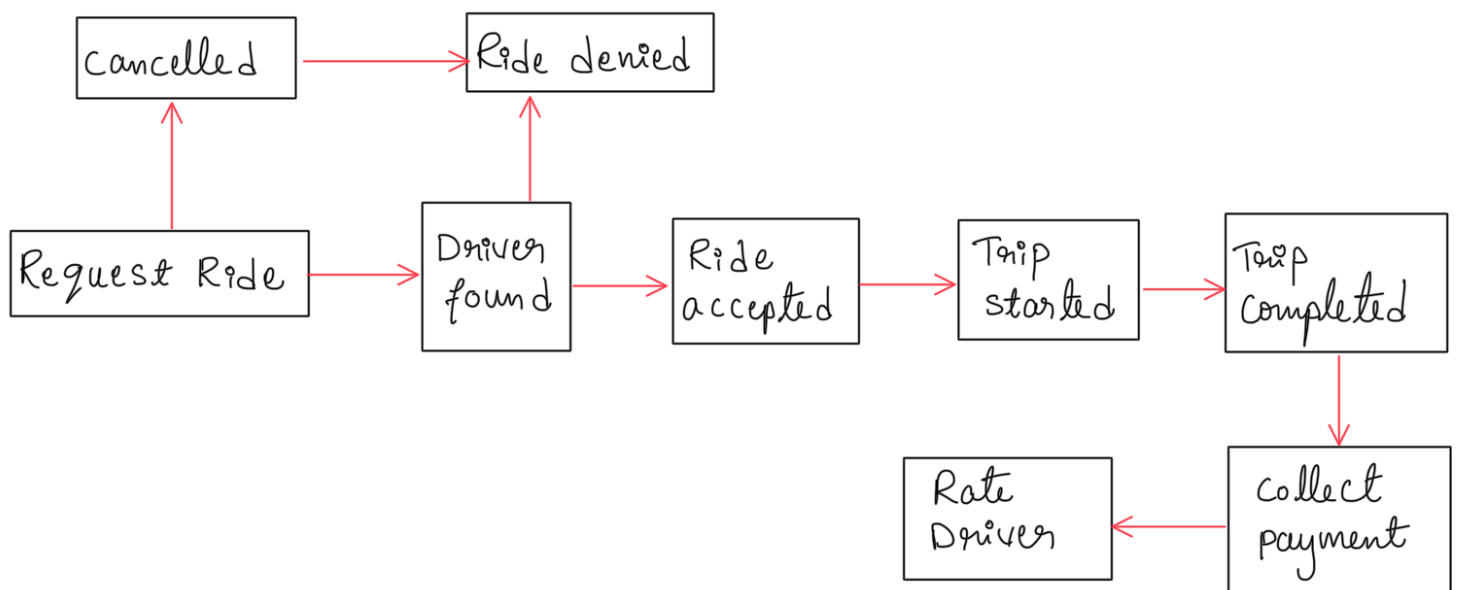


Exploring System Design of UBER part-1

DAY 113
31/10/25

Getting the hassle-free transportation Service (Uber, Ola) is Very Easy But is it also Simple to build these gigantic applications that have hundreds of Software Engineers working on them for a decade? Surely not. These Systems have much more complex architecture and there are a lot of components joined together internally to provide riding Services all over the world.



Requirements

Functional requirements

- Users should be able to see all the cabs available with minimum price and ETA
- Users should be able to book a cab for their destination
- Users should be able to see the location of the driver
- Users should be able to cancel their ride whenever they want

Non Functional Requirements

- High Available
- High Reliability
- Highly Scalable
- Low Latency

Capacity Estimation

Lets assume we have a 5 Million Active Users on our Application With 200,000 drivers and on an average there are 1 million rides daily, if a user performs 5 actions on an average then we need to handle 5 Million requests daily

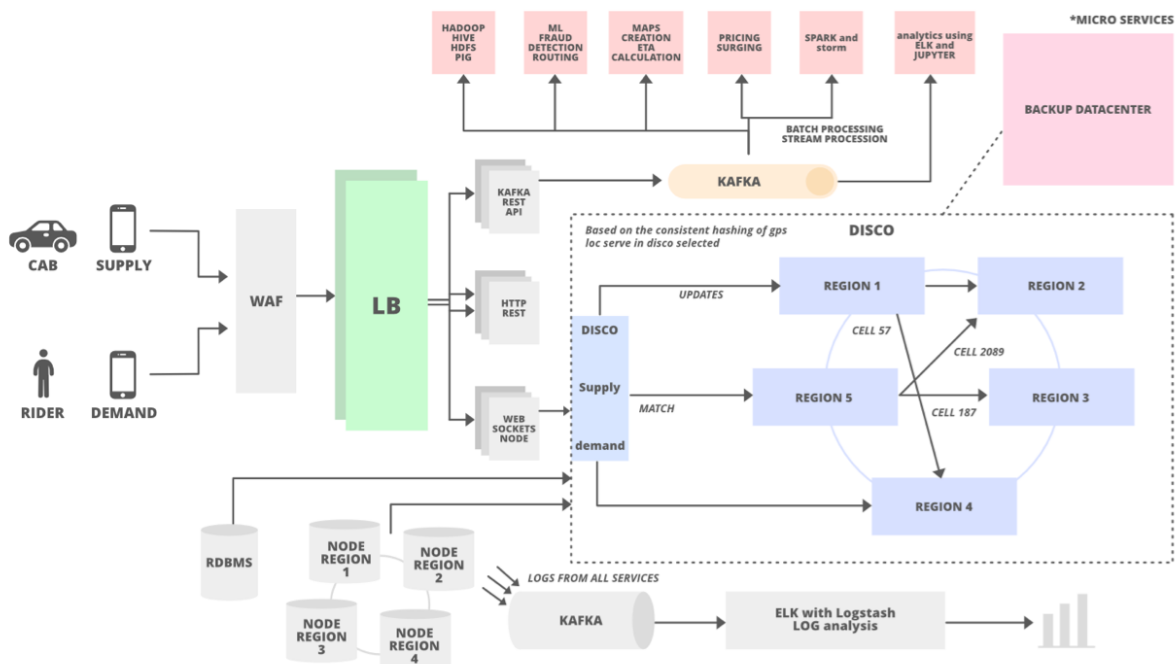
Requests per Second : ~ 58 RPS (5 million requests/day)

Daily storage Needed : ~ 2.32 GB (assuming 500 bytes per message)

UBER APP Low Level Design

We all are familiar with Uber services. A user can request a ride through the application and within a few minutes, a driver arrives nearby his/her location to take them to their destination.

- Earlier Uber was built on the "**monolithic**" software architecture model.
- They had a backend service, a frontend service, and a single database.
- They used Python and its frameworks and SQLAlchemy as the ORM layer to the database.
- This architecture was fine for a small number of trips in a few cities but when the service started expanding in other cities Uber team started facing the issue with the application.
- After the year 2014 Uber team decided to switch to the "**service-oriented architecture**" and now Uber also handles food delivery and cargo.



Talk about the challenges

one of the main task of UBER Service is to match the riders with cabs which means we need two different services in our architecture

- Supply Service - for cabs
- Demand Service - for Riders

Uber has a **Dispatch system** (Dispatch optimization/DISCO) in its architecture to match supply with demand. This dispatch system uses mobile phones and it takes the responsibility to match the drivers with riders (supply to demand).

How DISPATCH System Work ?

DISCO must have these goals...

- Reduce extra driving.
- Minimum waiting time
- Minimum overall ETA

The dispatch system completely works on maps and location data/GPS, so the first thing which is important is to model our maps and location data.

- Earth has a spherical shape so it's difficult to do summarization and approximation by using latitude and longitude. To solve this problem Uber uses the **Google S2 library**. This library divides the map data into tiny cells (for example 3km) and gives a unique ID to each cell. This is an easy way to spread data in the distributed system and store it easily.
- S2 library gives coverage for any given shape easily. Suppose you want to figure out all the supplies available within a 3km radius of a city.
- Using the S2 libraries you can draw a circle of 3km radius and it will filter out all the cells with IDs that lie in that particular circle.
- This way you can easily match the rider to the driver and you can easily find out the number of cars(supply) available in a particular region.